

Section 2: Data Preparation (14%) - Study Guide

Disclaimer based on your instructions: My automated web search tool primarily surfaced third-party aggregators (Medium, GitHub, etc.) rather than direct `docs.databricks.com` URLs for some of these specific exam phrasing exact matches. Therefore, as requested, I am explicitly stating this limitation. I have constructed this guide based on my knowledge of current Databricks products. For each section, I have provided the specific search paths and Databricks features you should look up on `docs.databricks.com` to verify these concepts.

Terminology Breakdown

Before diving into the exam objectives, here is a breakdown of the key terminology used in this section, grounded in the Databricks ecosystem:

- **Chunking / Chunking Strategy:** The process of breaking down large documents into smaller, manageable text segments (chunks). This is necessary because embedding models and LLMs have finite "context windows" (token limits).
 - **Vector Database / Databricks Mosaic AI Vector Search:** A database optimized for storing and retrieving high-dimensional vectors (embeddings). Databricks provides a fully managed vector search service integrated with Unity Catalog.
 - **Embedding Model:** An AI model (like BGE, OpenAI's text-embedding-ada-002, or Databricks foundational models) that converts text into a numerical array (vector) capturing its semantic meaning.
 - **Delta Lake:** An open-source storage layer that brings reliability (ACID transactions) to data lakes. In Databricks, all Unity Catalog managed tables are Delta tables by default.
 - **Re-ranking (or Reranker):** A secondary search step. After an initial, fast vector search returns a "rough" list of candidate documents (e.g., top 50), a reranker model (often a cross-encoder) precisely scores and re-orders them to ensure the most relevant results are at the top.
 - **MLflow Tracing:** A Databricks/MLflow feature that records the step-by-step execution path of a GenAI application, capturing intermediate steps like the exact documents retrieved by a Retriever.
 - **Retrieval Metrics (Precision@k, Recall@k, nDCG):** Standard information retrieval metrics used to quantify how well your search system is fetching relevant documents compared to a ground-truth dataset.
-

1. Apply a chunking strategy for a given document structure and model constraints

Concept: Documents must be broken into pieces (chunks) that fit into an embedding model's token limit. The strategy you apply depends on how the document is structured (e.g., Markdown, HTML, plain text) and the specific token constraints of the model.

Databricks Context & Implementation: If you have a strict model constraint (e.g., max 512 tokens), you must ensure no chunk exceeds this limit, otherwise, the text gets truncated and data is lost. * **Fixed-size chunking:** Splitting by a strict character or token count with an overlap (e.g., 500 characters, 50 overlap) is fast and strictly adheres to model constraints, but may split sentences mid-thought. * **Recursive chunking:** Tries to split on paragraphs first, then sentences, then words, keeping semantic units intact while still respecting the max constraint.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Data preparation for RAG applications"

2. Filter extraneous content in source documents that degrades quality of a RAG application

Concept: Raw documents often contain noise—headers, footers, navigation menus, boilerplate legal disclaimers, or ads. This "extraneous content" dilutes the semantic meaning of a chunk and confuses the LLM.

Databricks Context & Implementation: Before chunking and embedding, you must filter this out. On Databricks, this is typically done using Apache Spark or Python text processing libraries in a notebook or Delta Live Tables pipeline. * **Example:** Using BeautifulSoup to strip HTML tags, or writing Regex to remove standard company footers from all PDF pages before the text is saved to a Delta table.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Generative AI data preparation"

3. Choose the appropriate Python package to extract document content from provided source data and format

Concept: Source data comes in many formats. You must select the right open-source Python library to extract text reliably depending on the format.

Common Packages Used in Databricks: * `unstructured` : A highly recommended library for RAG that handles PDFs, Word, HTML, and extracts them into structured "elements" (titles, text, tables). * `PyPDF2` or `PyMuPDF (fitz)` : Useful for standard, simple PDF text extraction. * `beautifulsoup4 (bs4)` : The

standard for parsing HTML and extracting text from web pages while stripping tags. * **pytesseract** : Used for Optical Character Recognition (OCR) when dealing with scanned images or image-based PDFs.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "RAG reference architecture data preparation"

4. Define operations and sequence to write given chunked text into Delta Lake tables in Unity Catalog

Concept: Databricks Vector Search automatically syncs with a source Delta table. You must know the exact sequence of operations to prepare this table.

Databricks Context & Implementation: 1. **Extract & Chunk:** Use Python/Spark to extract text and apply your chunking strategy. 2. **Format DataFrame:** Ensure your Spark DataFrame has a primary key column (e.g., `id`) and a text column containing the chunk. 3. **Write to Delta:** Write the DataFrame to a Unity Catalog Delta table: `df.write.format("delta").saveAsTable("catalog.schema.chunked_docs")`. 4. **Enable Change Data Feed (CDF):** Vector Search requires CDF to sync updates. Run `ALTER TABLE catalog.schema.chunked_docs SET TBLPROPERTIES (delta.enableChangeDataFeed = true)`. 5. **Create Index:** Create the Vector Search Index pointing to this Delta table.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Create a Mosaic AI Vector Search index" * "Enable Change Data Feed"

5. Identify needed source documents that provide necessary knowledge and quality for a given RAG application

Concept: A RAG system is only as good as its knowledge base. You must identify which documents actually contain the answers your application is expected to provide, and ensure they are of high quality.

Databricks Context & Implementation: This is a data curation step. If building an IT Support bot, you must identify and include approved troubleshooting guides and exclude outdated manuals or unrelated marketing material. High-quality RAG requires identifying the "golden dataset."

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Generative AI data preparation best practices"

6. Use tools and metrics to evaluate retrieval performance

Concept: You must quantitatively measure if your vector search is actually returning the right chunks for a given query.

Databricks Context & Implementation: * **MLflow Evaluate:** MLflow provides built-in GenAI evaluation tools. * **Metrics:** You evaluate metrics like `RetrievalRelevance` (is the chunk relevant to the query?). You can also use standard metrics like Precision@k and Recall@k. * **MLflow Tracing:** To evaluate retrieval, MLflow needs to see *what* was retrieved. You decorate your retrieval function with `@mlflow.trace(span_type="RETRIEVER")` so MLflow can capture the inputs/outputs of the search step and run LLM judges against it.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "MLflow evaluate for RAG" * "MLflow Tracing"

7. Design retrieval systems using advanced chunking strategies

Concept: Moving beyond basic fixed-size splitting to strategies that preserve deeper semantic meaning and relationships in complex documents.

Databricks Context & Implementation: * **Document-Aware Chunking:** Using the structure of the document (Markdown headers, HTML sections) to keep related information together instead of arbitrarily cutting it. * **Parent-Child (Hierarchical) Chunking:** Creating small embeddings (child chunks) for highly precise vector search, but retrieving the larger surrounding context (parent chunk) to feed to the LLM. This solves the problem where a small chunk is easy to find but lacks enough context for the LLM to generate a good answer.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Advanced chunking strategies RAG"

8. Explain the role of re-ranking in the information retrieval process

Concept: Standard Vector Search prioritizes *speed* over absolute *precision*. Re-ranking is a second pass that fixes this by scoring the initial results much more rigorously.

Databricks Context & Implementation: 1. **Stage 1 (Retrieval):** The Databricks Vector Search index quickly pulls a large pool of candidate documents (e.g., top 50) using fast Approximate Nearest Neighbor algorithms. 2. **Stage 2 (Re-ranking):** A specialized, highly precise model (like a cross-encoder) scores the exact relationship between the user's specific query and each of those 50 documents, sorting them so the absolute best matches are at positions 1, 2, and 3. This improves the accuracy of the context fed to the LLM and reduces hallucinations.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Databricks Vector Search reranking" * "Foundation Model APIs cross-encoders"